ASSIGNMENT 1

PROBLEM STATEMENT : To set up and test the Arduino IDE for programming Arduino Uno and ESP32 boards.

CODE :

```
// Arduino Uno Blink Program
// Purpose: To verify that the Arduino Uno is working correctly
// and that the IDE, board selection, and COM port are configured properly.
void setup()
{
      // Configure built-in LED connected to digital pin 13 as OUTPUT
pinMode(LED_BUILTIN, OUTPUT);
}

void loop()
{
  // Turn ON the built-in LED
 digitalWrite(LED_BUILTIN, HIGH);

  // Wait for 1 second (1000 milliseconds)
delay(1000);

  // Turn OFF the built-in LED
digitalWrite(LED_BUILTIN, LOW);

  // Wait for 1 second
delay(1000);
}
```

OUTPUT :

The LED will turn ON for 1 second and OFF for 1 second continuously

ASSIGNMENT 2

PROBLEM STATEMENT : To interface an LED with Arduino Uno and toggle it first using software delay and then using a non-blocking timing method based on millis().

CODE :

PART A :- LED toggling using Software Delay

```
// Experiment: LED toggling using software delay
// Board: Arduino Uno
// LED: Built-in LED connected to digital pin 13
const int ledPin = 13;
void setup()
{
  // Configure LED pin as an output
pinMode(ledPin, OUTPUT);
}

void loop()
{
  // Turn LED ON
digitalWrite(ledPin, HIGH);

  // Wait for 1 second
delay(1000);

   // Turn LED OFF
digitalWrite(ledPin, LOW);

  // Wait for 1 second
delay(1000);
}
```

PART B :- LED toggling using Non-Blocking millis()

```
// Experiment: LED toggling using millis()
// Board: Arduino Uno
// This method does not block the execution of the program.
const int ledPin = 13;
  // Stores the time at which the LED was last toggled
unsigned long previousMillis = 0;

  // Time interval for LED toggling
const unsigned long interval = 1000;

  // Stores the current state of the LED
bool ledState = LOW;

void setup()
{
// Configure LED pin as an output
pinMode(ledPin, OUTPUT);
}

void loop()
{
// Get the current time in milliseconds
unsigned long currentMillis = millis();

// Check whether 1 second has passed
if (currentMillis - previousMillis >= interval)
{
  // Save the current time
previousMillis = currentMillis;
  // Toggle the LED state
 ledState = !ledState;
  // Apply the new state to the LED
```

```
digitalWrite(ledPin, ledState);
}
  // Other tasks can be executed here
  // because millis() does not block the program.
}
```

OUTPUT :

The LED will turn ON for 1 second and OFF for 1 second continuously in both programs

ASSIGNMENT 3

PROBLEM STATEMENT : To interface a pushbutton switch and an LED with Arduino Uno and control the LED using a simple delay-based debounce method.

CODE :

```
// Experiment: Pushbutton and LED with Debouncing
// Board: Arduino Uno
// Button: Digital Pin 2
// LED: Digital Pin 13
const int buttonPin = 2;  // Pushbutton connected to D2
const int ledPin = 13;     // LED connected to D13

void setup()
{
      // Configure the button pin as INPUT with internal pull-up resistor
pinMode(buttonPin, INPUT_PULLUP);

  // Configure the LED pin as OUTPUT
pinMode(ledPin, OUTPUT);

  // Initially turn OFF the LED
digitalWrite(ledPin, LOW);
}

void loop()
{
  // Read the current state of the pushbutton
int buttonState = digitalRead(buttonPin);
  // With INPUT_PULLUP, LOW means the button is pressed
if (buttonState == LOW)
  {
    // Wait for 50 ms to allow switch bouncing to settle
```

```
delay(50);
   // Read the button again after the debounce delay
buttonState = digitalRead(buttonPin);
   // Check whether the button is still pressed
if (buttonState == LOW)
  {
   // Turn ON the LED
digitalWrite(ledPin, HIGH);
  }
}
else
 {
  // Button is released, so turn OFF the LED
digitalWrite(ledPin, LOW);
  }
}
```

OUTPUT :

- When the pushbutton is released → LED remains OFF.
- When the pushbutton is pressed → After approximately 50 ms debounce, the LED turns ON.
- When the pushbutton is released again → LED turns OFF.

ASSIGNMENT 4

PROBLEM STATEMENT : To interface a 16×2 LCD with an I²C backpack to Arduino Uno and display text messages using I²C communication.

CODE :

```
// Experiment: 16x2 I2C LCD
// Board: Arduino Uno
// Display: Hello World
#include <Wire.h>                       // I2C communication library
#include <LiquidCrystal_I2C.h>          // I2C LCD library

// Create LCD object
// 0x27 = common I2C address
// 16 = number of columns
// 2 = number of rows
LiquidCrystal_I2C lcd(0x27, 16, 2);
void setup()
{
  // Initialize the LCD
lcd.init();

  // Turn ON the LCD backlight
lcd.backlight();

  // Display "Hello World"
lcd.setCursor(0, 0);                    // Column 0, Row 0
lcd.print("Hello World");
}
void loop()
{
  // Nothing required here
}
```

OUTPUT :

The 16×2 LCD displays “Hello World” on the first line of the screen. The second line remains blank.

ASSIGNMENT 6

PROBLEM STATEMENT : To generate PWM signals using Arduino Uno and study their use for LED brightness control, buzzer drive control, and DC motor speed control.

CODE :

```
// Experiment: PWM Control
// Board: Arduino Uno
// LED       →  Pin 9
// Buzzer    →  Pin 10
// DC Motor  →  Pin 11 through L293D motor driver
const int ledPin = 9;
const int buzzerPin = 10;
const int motorPin = 11;

void setup()
{
  // Set PWM pins as outputs
pinMode(ledPin, OUTPUT);
pinMode(buzzerPin, OUTPUT);
pinMode(motorPin, OUTPUT);
}
void loop()
{
  // Generate PWM values from 0 to 255
for (int pwmValue = 0; pwmValue <= 255; pwmValue++)
{
  // Control LED brightness
analogWrite(ledPin, pwmValue);

  // Control buzzer drive level
analogWrite(buzzerPin, pwmValue);
```

```
  // Control DC motor speed
analogWrite(motorPin, pwmValue);

  // Small delay to observe the change
delay(20);
}
   // Decrease PWM value from 255 to 0
for (int pwmValue = 255; pwmValue >= 0; pwmValue--)
  {
     // Control LED brightness
analogWrite(ledPin, pwmValue);

     // Control buzzer drive level
analogWrite(buzzerPin, pwmValue);

     // Control DC motor speed
analogWrite(motorPin, pwmValue);
delay(20);
  }
}
```

OUTPUT :

The LED brightness gradually increases and then decreases. The buzzer drive level changes with the PWM signal, and the DC motor speed gradually increases and decreases. This cycle repeats continuously.

ASSIGNMENT 7

PROBLEM STATEMENT : To measure the period and frequency of a square-wave signal generated by an external function generator using Arduino Uno.

CODE :

```
// Experiment: Measurement of Period and Frequency
// Board: Arduino Uno
// External square-wave signal → Digital Pin 2
const int signalPin = 2;

unsigned long highTime;
unsigned long lowTime;
unsigned long period;

float frequency;

void setup()
{
  // Configure D2 as input
pinMode(signalPin, INPUT);

  // Start Serial Monitor
Serial.begin(9600);
}

void loop()
{
  // Measure the time for which the signal is HIGH
highTime = pulseIn(signalPin, HIGH);

  // Measure the time for which the signal is LOW
```

```
lowTime = pulseIn(signalPin, LOW);
  // Total time of one complete cycle
  // Period = HIGH time + LOW time
period = highTime + lowTime;
  // Convert period from microseconds to frequency in Hz
  // 1 second = 1,000,000 microseconds
if (period > 0)
 {
frequency = 1000000.0 / period;
  // Display results on Serial Monitor
Serial.print("Period = ");
Serial.print(period);
Serial.println(" microseconds");
Serial.print("Frequency = ");
Serial.print(frequency);
Serial.println(" Hz");
Serial.println("--------------------");
  }
  // Small delay before next measurement
delay(500);
}
```

OUTPUT :

The Serial Monitor displays the measured time period in microseconds and the corresponding frequency in Hertz (Hz) of the square-wave signal.